Ethereum: A Secure Decentralised Generalised Transaction Ledger (Shangai Version)

Dr. Gavin Wood
Iker Martín Canut

Abril 2023
Diciembre 2025

1. Idea Central

1. Idea Central

- Ethereum es una **máquina de estado distribuida** basada en transacciones.

1. Idea Central

- Ethereum es una **máquina de estado distribuida** basada en transacciones.
- El consenso asegura una historia única por más de que no haya confianza entre las partes.

1. Idea Central

- Ethereum es una **máquina de estado distribuida** basada en transacciones.
- El consenso asegura una historia única por más de que no haya confianza entre las partes.
- Generaliza el paradigma introducido por Bitcoin.

El Paradigma de la Cadena de Bloques

2. El Paradigma de la Cadena de Bloques

2. El Paradigma de la Cadena de Bloques

- Una transacción (T) representa un arco **válido** entre dos estados σ_t y σ_{t+1}:

$$\sigma_{t+1} \equiv \Upsilon(\sigma_t, T)$$

2. El Paradigma de la Cadena de Bloques

- Una transacción (T) representa un arco **válido** entre dos estados σ_t y σ_{t+1}:

$$\sigma_{t+1} \equiv \Upsilon(\sigma_t, T)$$

- La función de transición de estado (Υ) y el estado (σ) permiten **computación arbitraria** y **almacenamiento arbitrario de estado** entre transacciones.

2. El Paradigma de la Cadena de Bloques

- Una transacción (T) representa un arco **válido** entre dos estados σ_t y σ_{t+1}:

$$\sigma_{t+1} \equiv \Upsilon(\sigma_t, T)$$

- La función de transición de estado (Υ) y el estado (σ) permiten **computación arbitraria** y **almacenamiento arbitrario de estado** entre transacciones.
- Las transacciones se agrupan en bloques, encadenados por *hashes* criptográficos. Los bloques actúan como un diario de transacciones y el estado final.

2.2. Historia

2.2. Historia

- La estructura resultante de bloques es un **árbol** debido a la naturaleza descentralizada.

2.2. Historia

- La estructura resultante de bloques es un **árbol** debido a la naturaleza descentralizada.
- Un desacuerdo resultaría en una **bifurcación** (*fork*), donde múltiples estados coexistirían.

2.2. Historia

- La estructura resultante de bloques es un **árbol** debido a la naturaleza descentralizada.
- Un desacuerdo resultaría en una **bifurcación** (*fork*), donde múltiples estados coexistirían.
- **Consenso Post-Paris:** Desde el *hard fork* de Paris, el consenso es gestionado por el protocolo **Beacon Chain** (la capa de consenso), que decide la historia canónica.

Bloques, Estado y Transacciones

4. Bloques, Estado y Transacciones

4. Bloques, Estado y Transacciones

- El estado mundial no se almacena en la cadena de bloques, sino que se mantiene en un **árbol Merkle Patricia modificado (trie)**.

4. Bloques, Estado y Transacciones

- El estado mundial no se almacena en la cadena de bloques, sino que se mantiene en un **árbol Merkle Patricia modificado (trie)**.
- Las hojas guardan valores y sus hashes. Sus padres también contiene hashes de sus hijos. Asi hasta la raíz.

4. Bloques, Estado y Transacciones

- El estado mundial no se almacena en la cadena de bloques, sino que se mantiene en un **árbol Merkle Patricia modificado (trie)**.
- Las hojas guardan valores y sus hashes. Sus padres también contiene hashes de sus hijos. Asi hasta la raíz.
- La raíz del trie proporciona una **identidad segura** para todo el estado del sistema.

4.1. Estado Mundial

4.1. Estado Mundial

El estado de cuenta para una dirección *a* consta de cuatro campos:

4.1. Estado Mundial

El estado de cuenta para una dirección a consta de cuatro campos:

1. **nonce ($\sigma[a]_n$):** Número de transacciones enviadas o creaciones de contratos.

4.1. Estado Mundial

El estado de cuenta para una dirección a consta de cuatro campos:

1. **nonce ($\sigma[a]_n$):** Número de transacciones enviadas o creaciones de contratos.
2. **balance ($\sigma[a]_b$):** Valor escalar de Wei poseído.

4.1. Estado Mundial

El estado de cuenta para una dirección a consta de cuatro campos:

1. **nonce ($\sigma[a]_n$):** Número de transacciones enviadas o creaciones de contratos.
2. **balance ($\sigma[a]_b$):** Valor escalar de Wei poseído.
3. **storageRoot ($\sigma[a]_s$):** *Hash* de la raíz del trie Merkle Patricia que codifica el almacenamiento de la cuenta (un mapeo de enteros de 256 bits).

4.1. Estado Mundial

El estado de cuenta para una dirección a consta de cuatro campos:

1. **nonce ($\sigma[a]_n$):** Número de transacciones enviadas o creaciones de contratos.
2. **balance ($\sigma[a]_b$):** Valor escalar de Wei poseído.
3. **storageRoot ($\sigma[a]_s$):** *Hash* de la raíz del trie Merkle Patricia que codifica el almacenamiento de la cuenta (un mapeo de enteros de 256 bits).
4. **codeHash ($\sigma[a]_c$):** *Hash* del código EVM asociado. Si es el *hash* de la cadena vacía, es una cuenta **simple** (no contrato).

4.2. La Transacción

4.2. La Transacción

- Una transacción (T) es una instrucción única firmada por un actor externo (no puede ser un contrato).

4.2. La Transacción

- Una transacción (T) es una instrucción única firmada por un actor externo (no puede ser un contrato).
- Existen distintos tipos de transacciones.

4.2. La Transacción

- Una transacción (T) es una instrucción única firmada por un actor externo (no puede ser un contrato).
- Existen distintos tipos de transacciones.
- **Campos Comunes:** T_x (tipo), T_n (*nonce*), T_g (*gasLimit*), T_t (destinatario), T_v (valor) y T_r, T_s (valores de firma).

4.2. La Transacción

- Una transacción (T) es una instrucción única firmada por un actor externo (no puede ser un contrato).
- Existen distintos tipos de transacciones.
- **Campos Comunes:** T_x (tipo), T_n (*nonce*), T_g (*gasLimit*), T_t (destinatario), T_v (valor) y T_r, T_s (valores de firma).
- El costo se divide en 2, una parte se quema y la tarifa de prioridad va al validador.

4.3. Retiros de la Capa de Consenso (Post-Shanghái)

4.3. Retiros de la Capa de Consenso (Post-Shanghái)

- Un retiro (W) es una tupla de datos que describe el retiro de Ether apostado por un validador de la capa de consenso.

4.3. Retiros de la Capa de Consenso (Post-Shanghái)

- Un retiro (W) es una tupla de datos que describe el retiro de Ether apostado por un validador de la capa de consenso.
- Es creado y validado en la capa de consenso y empujado a la capa de ejecución.

4.3. Retiros de la Capa de Consenso (Post-Shanghái)

- Un retiro (W) es una tupla de datos que describe el retiro de Ether apostado por un validador de la capa de consenso.
- Es creado y validado en la capa de consenso y empujado a la capa de ejecución.
- **Campos:** W_g (*globalIndex*), W_v (*validatorIndex*), W_r (*recipient*) y W_a (*amount*, en Gwei).

4.4. El Bloque

4.4. El Bloque

Un bloque (B) es la colección de:

4.4. El Bloque

Un bloque (B) es la colección de:

1. El Encabezado del Bloque (H): que contiene hashes, dirección del validador, hash del trie final, raices de los tries de transacciones, ...

4.4. El Bloque

Un bloque (B) es la colección de:

1. El Encabezado del Bloque (H): que contiene hashes, dirección del validador, hash del trie final, raices de los tries de transacciones, ...
2. Las transacciones (B_T).

4.4. El Bloque

Un bloque (B) es la colección de:

1. El Encabezado del Bloque (H): que contiene hashes, dirección del validador, hash del trie final, raices de los tries de transacciones, ...
2. Las transacciones (B_T).
3. Los retiros (B_W) (desde el *hard fork* de Shanghái).

4.4.2. Validez Holística

4.4.2. Validez Holística

- La validez de un bloque se afirma si y solo si el campo de *ommers* (B_U) es una matriz vacía y el encabezado (H) es consistente con las transacciones (B_T) y los retiros (B_W).

4.4.2. Validez Holística

- La validez de un bloque se afirma si y solo si el campo de *ommers* (B_U) es una matriz vacía y el encabezado (H) es consistente con las transacciones (B_T) y los retiros (B_W).
- El *stateRoot* (H_r) debe coincidir con el estado resultante después de ejecutar todas las transacciones y retiros en orden.

4.4.2. Validez Holística

- La validez de un bloque se afirma si y solo si el campo de *ommers* (B_U) es una matriz vacía y el encabezado (H) es consistente con las transacciones (B_T) y los retiros (B_W).
- El *stateRoot* (H_r) debe coincidir con el estado resultante después de ejecutar todas las transacciones y retiros en orden.
- Las raíces de los tries de transacciones, recibos y retiros también deben derivarse correctamente.

Gas y Pago

5. Gas y Pago

5. Gas y Pago

- Toda computación programable está sujeta a **tarifas de gas** para evitar el abuso de la red.

5. Gas y Pago

- Toda computación programable está sujeta a **tarifas de gas** para evitar el abuso de la red.
- **Gas Limit (T_g):** Cantidad máxima de gas que se compra por adelantado.

5. Gas y Pago

- Toda computación programable está sujeta a **tarifas de gas** para evitar el abuso de la red.
- **Gas Limit (T_g):** Cantidad máxima de gas que se compra por adelantado.
- **Base Fee (H_f)**

5. Gas y Pago

- Toda computación programable está sujeta a **tarifas de gas** para evitar el abuso de la red.
- **Gas Limit (T_g):** Cantidad máxima de gas que se compra por adelantado.
- **Base Fee (H_f)**
- **Priority Fee (Tarifa de Prioridad)**.

5. Gas y Pago

- Toda computación programable está sujeta a **tarifas de gas** para evitar el abuso de la red.
- **Gas Limit (T_g):** Cantidad máxima de gas que se compra por adelantado.
- **Base Fee (H_f)**
- **Priority Fee (Tarifa de Prioridad)**.
- El gas es un mecanismo de seguridad, no solo de pago. Limita la computación y previene loops infinitos, ataques de denegación de servicio.

Ejecución de Transacciones

6. Ejecución de Transacciones

6. Ejecución de Transacciones

- Una transacción debe pasar pruebas de **validez intrínseca** (firma válida, *nonce* correcto, balance suficiente para el costo inicial, y maxFeePerGas o gasPrice debe ser $\geq$ blockBaseFee).

6. Ejecución de Transacciones

- Una transacción debe pasar pruebas de **validez intrínseca** (firma válida, *nonce* correcto, balance suficiente para el costo inicial, y maxFeePerGas o gasPrice debe ser $\geq$ blockBaseFee).
- Se crea un subestado A con logs, cuentas que participan, información de contratos, ...

6. Ejecución de Transacciones

- Una transacción debe pasar pruebas de **validez intrínseca** (firma válida, *nonce* correcto, balance suficiente para el costo inicial, y maxFeePerGas o gasPrice debe ser $\geq$ blockBaseFee).
- Se crea un subestado A con logs, cuentas que participan, información de contratos, ...
- Se definen los costos.

6. Ejecución de Transacciones

- Una transacción debe pasar pruebas de **validez intrínseca** (firma válida, *nonce* correcto, balance suficiente para el costo inicial, y maxFeePerGas o gasPrice debe ser $\geq$ blockBaseFee).
- Se crea un subestado A con logs, cuentas que participan, información de contratos, ...
- Se definen los costos.
- Se ejecutan las transacciones.

6. Ejecución de Transacciones

- Una transacción debe pasar pruebas de **validez intrínseca** (firma válida, *nonce* correcto, balance suficiente para el costo inicial, y maxFeePerGas o gasPrice debe ser $\geq$ blockBaseFee).
- Se crea un subestado A con logs, cuentas que participan, información de contratos, ...
- Se definen los costos.
- Se ejecutan las transacciones.
- Se incrementan los nonces.

6. Ejecución de Transacciones

- Una transacción debe pasar pruebas de **validez intrínseca** (firma válida, *nonce* correcto, balance suficiente para el costo inicial, y maxFeePerGas o gasPrice debe ser $\geq$ blockBaseFee).
- Se crea un subestado A con logs, cuentas que participan, información de contratos, ...
- Se definen los costos.
- Se ejecutan las transacciones.
- Se incrementan los nonces.
- Se calcula el gas reembolsable y se da la tarifa al validador.

7. Creación de Contratos

7. Creación de Contratos

- Es un programa autoejecutable almacenado en la blockchain.

7. Creación de Contratos

- Es un programa autoejecutable almacenado en la blockchain.
- La función Λ define la creación de un nuevo contrato, creando una nueva cuenta con una nueva dirección y ejecutando el código de inicialización.

7. Creación de Contratos

- Es un programa autoejecutable almacenado en la blockchain.
- La función Λ define la creación de un nuevo contrato, creando una nueva cuenta con una nueva dirección y ejecutando el código de inicialización.
- Si ocurre una excepción de **falta de gas (OOG)** en la inicialización, el estado revierte al punto anterior a la creación.

7. Creación de Contratos

- Es un programa autoejecutable almacenado en la blockchain.
- La función Λ define la creación de un nuevo contrato, creando una nueva cuenta con una nueva dirección y ejecutando el código de inicialización.
- Si ocurre una excepción de **falta de gas (OOG)** en la inicialización, el estado revierte al punto anterior a la creación.
- Si la inicialización es exitosa, se paga un costo final proporcional al tamaño del código resultante ($\mathbf{o}$).

7. Creación de Contratos

- Es un programa autoejecutable almacenado en la blockchain.
- La función Λ define la creación de un nuevo contrato, creando una nueva cuenta con una nueva dirección y ejecutando el código de inicialización.
- Si ocurre una excepción de **falta de gas (OOG)** en la inicialización, el estado revierte al punto anterior a la creación.
- Si la inicialización es exitosa, se paga un costo final proporcional al tamaño del código resultante ($\mathbf{o}$).
- Si todo es exitoso, el nuevo estado persiste.

8. Llamada de Mensaje

8. Llamada de Mensaje

- La función de llamada de mensaje define cómo una cuenta interactúa con otra (o consigo misma) para ejecutar código.

8. Llamada de Mensaje

- La función de llamada de mensaje define cómo una cuenta interactúa con otra (o consigo misma) para ejecutar código.
- La función Θ requiere parámetros como el remitente (s), el destinatario (r), el gas (g), el valor (v) y los datos de entrada (d).

8. Llamada de Mensaje

- La función de llamada de mensaje define cómo una cuenta interactúa con otra (o consigo misma) para ejecutar código.
- La función Θ requiere parámetros como el remitente (s), el destinatario (r), el gas (g), el valor (v) y los datos de entrada (d).
- **Contratos Precompilados:** Las direcciones 1 a 9 se reservan para contratos precompilados que ejecutan funciones especializadas fuera del marco general de la EVM (Ξ).

Modelo de Ejecución

9. Modelo de Ejecución

9. Modelo de Ejecución

- El modelo es gobernado por la **EVM**, una máquina cuasi-Turing-completa limitada por el parámetro **gas**.

9. Modelo de Ejecución

- El modelo es gobernado por la **EVM**, una máquina cuasi-Turing-completa limitada por el parámetro **gas**.
- Arquitectura simple basada en pilas, con tamaño de palabra de **256b** y un límite de pila de **1024**. No sigue arquitectura von Neumann (código en ROM).

9. Modelo de Ejecución

- El modelo es gobernado por la **EVM**, una máquina cuasi-Turing-completa limitada por el parámetro **gas**.
- Arquitectura simple basada en pilas, con tamaño de palabra de **256b** y un límite de pila de **1024**. No sigue arquitectura von Neumann (código en ROM).
- **Memoria**: Volátil, matriz de bytes direccionada por palabras.

9. Modelo de Ejecución

- El modelo es gobernado por la **EVM**, una máquina cuasi-Turing-completa limitada por el parámetro **gas**.
- Arquitectura simple basada en pilas, con tamaño de palabra de **256b** y un límite de pila de **1024**. No sigue arquitectura von Neumann (código en ROM).
- **Memoria**: Volátil, matriz de bytes direccionada por palabras.
- **Almacenamiento no volátil**: parte del estado del sistema, direccionable por palabras de 256 bits.

9. Modelo de Ejecución

- El modelo es gobernado por la **EVM**, una máquina cuasi-Turing-completa limitada por el parámetro **gas**.
- Arquitectura simple basada en pilas, con tamaño de palabra de **256b** y un límite de pila de **1024**. No sigue arquitectura von Neumann (código en ROM).
- **Memoria**: Volátil, matriz de bytes direccionada por palabras.
- **Almacenamiento no volátil**: parte del estado del sistema, direccionable por palabras de 256 bits.
- Las ejecuciones excepcionales (ej. falta de gas, instrucción inválida) detienen la máquina inmediatamente.

9. Modelo de Ejecución

- El modelo es gobernado por la **EVM**, una máquina cuasi-Turing-completa limitada por el parámetro **gas**.
- Arquitectura simple basada en pilas, con tamaño de palabra de **256b** y un límite de pila de **1024**. No sigue arquitectura von Neumann (código en ROM).
- **Memoria**: Volátil, matriz de bytes direccionada por palabras.
- **Almacenamiento no volátil**: parte del estado del sistema, direccionable por palabras de 256 bits.
- Las ejecuciones excepcionales (ej. falta de gas, instrucción inválida) detienen la máquina inmediatamente.
- Define formalmente como se ejecutan las instrucciones.

Transición a Prueba de Participación y Finalización

10. Transición a Prueba de Participación

10. Transición a Prueba de Participación

- El *hard fork* de Paris cambió el mecanismo de consenso de PoW a **Prueba de Participación (PoS)**.

10. Transición a Prueba de Participación

- El *hard fork* de Paris cambió el mecanismo de consenso de PoW a **Prueba de Participación (PoS)**.
- Pasó de ser una seguridad basada en trabajo computacional a una basada en la participación económica.

10. Transición a Prueba de Participación

- El *hard fork* de Paris cambió el mecanismo de consenso de PoW a **Prueba de Participación (PoS)**.
- Pasó de ser una seguridad basada en trabajo computacional a una basada en la participación económica.
- Menor consumo energético, mismas reglas de ejecución: La EVM es la misma.

13. Implementación de Contratos

13. Implementación de Contratos

- **Fuentes de Datos**: Un contrato proporciona acceso a información del mundo externo (ej. temperatura de Nueva York). Esto se implementa típicamente con un servidor externo.

13. Implementación de Contratos

- **Fuentes de Datos**: Un contrato proporciona acceso a información del mundo externo (ej. temperatura de Nueva York). Esto se implementa típicamente con un servidor externo.
- **Números Aleatorios**: Imposible en un sistema determinista. Se aproximan con números pseudo-aleatorios derivados de datos generalmente incognoscibles al momento de la transacción, como el hash del bloque.

Conclusión

Conclusión

- El protocolo Ethereum ha sido formalmente definido.

Conclusión

- El protocolo Ethereum ha sido formalmente definido.
- Permite implementar un nodo en la red y unirse a un sistema operativo social, descentralizado y seguro.

Conclusión

- El protocolo Ethereum ha sido formalmente definido.
- Permite implementar un nodo en la red y unirse a un sistema operativo social, descentralizado y seguro.
- Se pueden crear contratos para especificar algorítmicamente y hacer cumplir de manera autónoma las reglas de interacción.